

CO4 - AT1 – ARTICLE REVIEW

Name: Benita Johnson J

Reg no: 192412595

Course name: 5G/6G Communication systems

Course code: ECA5609

Topic: Edge AI for Intelligent 5G Network Optimization

ABSTRACT:

This paper proposes an Edge Intelligence-based approach for efficient computation offloading and resource allocation in 5G-Beyond networks. Deep Reinforcement Learning (DRL) is used to make intelligent decisions on whether computational tasks should be processed locally or offloaded to edge servers. A Deep Deterministic Policy Gradient (DDPG) based approach is developed to learn effective offloading and resource-allocation decisions in the dynamic network environment. The objective is to reduce energy consumption while satisfying computation-delay requirements.

TECHNICAL IMPLEMENTATION:

- Uses an end–edge–cloud architecture with mobile devices, Small-cell Base Stations (SBSs), a Macro Base Station (MBS), and cloud resources.
- The offloading problem is formulated as a Markov Decision Process (MDP) for intelligent decision-making.
- A Deep Deterministic Policy Gradient (DDPG) algorithm learns optimal computation-offloading and resource-allocation decisions.
- A weighted bipartite graph and Hungarian algorithm are used to refine the offloading decisions.
- The proposed approach is evaluated through Python and TensorFlow simulations.

CONCLUSION:

The study demonstrates that Edge Intelligence can effectively improve resource management in 5G-Beyond networks. The proposed DDPG-based approach learns suitable computation-offloading and resource-allocation decisions according to changing network conditions, achieving lower energy consumption while maintaining computation-delay requirements. Simulation results show that the proposed method performs better than the considered local-computing and full-offloading approaches, particularly for computation-intensive tasks. The work demonstrates the potential of combining Edge AI and 5G networks to provide intelligent, adaptive, and energy-efficient network optimization.

Edge Intelligence for Energy-efficient Computation Offloading and Resource Allocation in 5G Beyond

Yueyue Dai, *Member, IEEE*, Ke Zhang, Sabita Maharjan, *Senior Member, IEEE*, and Yan Zhang, *Fellow, IEEE*

Abstract—5G beyond is an end-edge-cloud orchestrated network that can exploit heterogeneous capabilities of the end devices, edge servers, and the cloud and thus has the potential to enable computation-intensive and delay-sensitive applications via computation offloading. However, in multi user wireless networks, diverse application requirements and the possibility of various radio access modes for communication among devices make it challenging to design an optimal computation offloading scheme. In addition, having access to complete network information that includes variables such as wireless channel state, and available bandwidth and computation resources, is a major issue. Deep Reinforcement Learning (DRL) is an emerging technique to address such an issue with limited and less accurate network information. In this paper, we utilize DRL to design an optimal computation offloading and resource allocation strategy for minimizing system energy consumption. We first present a multi-user end-edge-cloud orchestrated network where all devices and base stations have computation capabilities. Then, we formulate the joint computation offloading and resource allocation problem as a Markov Decision Process (MDP) and propose a new DRL algorithm to minimize system energy consumption. Numerical results based on a real-world dataset demonstrate that the proposed DRL-based algorithm significantly outperforms the benchmark policies in terms of system energy consumption. Extensive simulations show that learning rate, discount factor, and number of devices have considerable influence on the performance of the proposed algorithm.

Index Terms—Deep reinforcement learning, Edge computing, Computation offloading, 5G Beyond.

I. INTRODUCTION

Recent advancements in Internet of Things (IoT) and 5G wireless networks have paved a way towards realizing new application such as surveillance, augmented/virtual reality, and face recognition, which usually are both computation and caching resource intensive. For battery-powered and resourceconstrained devices, such as wearable devices, on-device sensors, and smart phones, it is therefore challenging to support such applications, in particular, delay sensitive applications [1]. To alleviate computation limitations and to prolong battery

Center for Digital Engineering, Norway. (email: sabita@ifi.uio.no, yanzhang@ieee.org).

lifetime of IoT devices, Mobile Cloud Computing (MCC) leverages offloading computation intensive tasks to the centralized cloud server. MCC however may result in high communication latency, low coverage, and lagged data transmission thus limiting its applicability for real-time applications such as automated driving and smart navigation. Edge computing is a promising paradigm to address such issues associated with MCC, by pushing additional computation resources available at the edge servers [2].

Utilizing computation offloading, IoT devices can offload their computation-intensive and delay-sensitive tasks to nearby distributed base stations, access points, and road-side units, integrated with edge servers [3]. There has been considerable amount of work focusing on computation offloading. The authors in [4] proposed to offload computation tasks to lightweight and distributed vehicular edge servers to minimize task processing latency. In [5] and [6], the authors proposed to offload computation tasks to the Macro-cell Base Stations (MBS) for minimizing the maximum weighted energy consumption. The authors in [7] proposed to offload computation tasks to multiple distributed Small-cell Base Stations (SBS) and the MBS, to minimize energy consumption. However, the computation capacities of these ubiquitous edge servers are often limited such that offloading all devices' tasks to edge servers may instead result in a higher computation latency. Moreover, computation offloading especially in ultra dense and heterogeneous 5G networks can lead to a higher task processing delay compared to local computing in devices, due to additional delay caused by the need to transmit the data to base stations. Local computing, on the other hand, can result in low task execution latency since in that case there is no delay added due to transmitting the task and the result of task execution back and forth.

To make full use of heterogeneous computation capabilities of devices, edge servers, and the cloud, 5G beyond networks offers an end-edge-cloud orchestrated architecture that can

Copyright (c) 2015 IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

This research is partially supported by the National Natural Science Foundation of China under Grant 61941102 and partially supported by the Xi'an Key Laboratory of Mobile Edge Computing and Security, under Grant No. 2018050522D3CG36.

Y. Dai and K. Zhang are with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (email: yueyuedai@ieee.org; zhangke@uestc.edu.cn).

S. Maharjan and Y. Zhang (corresponding author) are with Department of Informatics, University of Oslo, Norway, and also with Simula Metropolitan

concurrently support local computing and computation offloading. Optimizing computation offloading in this case means deciding whether to execute tasks locally at the end devices or offload them to edge servers or to offload a certain amount of them to the central server. However, due to the highly dynamic and stochastic nature of wireless systems, the lack of complete network information such as wireless channel state, available bandwidth and computation resources, makes it challenging to design an optimal offloading strategy. In addition, diverse application-specific requirements and the use of multiple radio access modes, make the problem sufficiently complex.

Edge intelligence is a promising paradigm that leverages capability at the network edge to enable real-time services [8], [9]. By integrating AI into edge networks, edge intelligent servers have full insight of working environments in terms of the resource correlation between heterogeneous types and the feasibility of cooperation with adjacent nodes [10]. Moreover, as edge services typically utilize edge resources to estimate the environment, edge resource scheduling does not require the information of the entire network from the central cloud. Transmitting locally learned information to the distributed AI processing entities is more efficient than obtaining resource scheduling strategies from remote central AI nodes [11]. Thus, diverse servers at the network edge can incorporate heterogeneous resources and offer flexible communication and computing services in an efficient manner.

In this work, we propose an edge intelligence empowered 5G beyond network by jointly optimizing computation offloading and scheduling the distributed resources. We consider an end-edge-cloud orchestrated network consisting of devices, edge computing servers and a cloud server where computing tasks can be executed locally with cooperation between the edge computing servers and the cloud server. Due to highly dynamic of wireless environment and the diverse application requirements, we exploit Deep Reinforcement Learning (DRL) to learn dynamic network topology and time-varying wireless channel condition and then design a joint computation offloading and resource allocation scheme. The key contributions of our work are summarized as follows:

- We present a multi-user end-edge-cloud orchestrated network where devices can offload computation-intensive and delay-sensitive tasks to a particular edge server on an SBS and/or the MBS, respectively.
- We formulate the joint computation offloading and resource allocation problem to minimize system energy consumption taking stringent delay constraints into account.
- We propose a DRL-based computation offloading and resource allocation algorithm to reduce energy consumption. Numerical results based on a real-world dataset demonstrate that the proposed DRL-based

algorithm significantly outperforms the benchmark policies.

The remainder of this paper is organized as follows. The related work is discussed in Section II. We then present the system model of the end-edge-cloud orchestrated network in Section III. The joint computation offloading and resource allocation problem is formulated in Section IV. We design a DRL-based algorithm to solve the proposed computation offloading problem in Section V. Numerical results are presented in Section VI. Finally, the paper is concluded in Section VII.

II. RELATED WORK

In this section, we first introduce the state-of-the-art researches on computation offloading in wireless networks and then discuss the work that exploits DRL for computation offloading.

A. Computation Offloading in Wireless Networks

In 5G beyond, devices are connected to the infrastructure through wireless communication. This introduces high agility in resource deployment and facilitates pushing edge services closer to the end users. By offloading tasks directly to edge servers, the resource burden of the devices can be alleviated, and the application processing latency can also be considerably reduced.

Computation offloading in wireless networks is widely investigated. For instance, the authors in [12] considered an ultra-dense network and proposed a two-tier greedy offloading scheme to minimize computation overhead. The authors in [13] proposed a computation offloading scheme to determine whether the computation-intensive tasks should be executed locally or remotely on the MEC servers. The authors in [5] considered a multi-user system based on time-division multiple access and orthogonal frequency-division multiple access, and formulated an computation offloading optimization problem to minimize the weighted energy consumption under the constraint on computation latency. In [14], the authors proposed a three-hierarchical offloading optimization strategy, which jointly optimizes bandwidth, offloading strategy to reduce the system latency and energy consumption. In [15], the authors proposed a multi-user MEC system by integrating a multi-antenna Access Point (AP) with an MEC server such that mobile users can execute their respective tasks locally or offload them to the AP. The authors in [7] proposed a computation offloading algorithm by jointly considering the offloading decision, computation resources, and the transmit power for computation offloading. The authors in [16] proposed an iterative computation offloading algorithm to improve the resource utilization efficiency in wireless networks.

The optimization algorithms of the above researches often require complete and accurate network information. However,

the wireless network is time-varying such that the accurate network information of wireless channel state, bandwidth and computation resources are difficult to obtain. Thus, the above optimization algorithms may not be practical in the wireless networks.

B. Deep Reinforcement Learning for Computation Offloading

Deep reinforcement learning is a promising technique to address the problems with uncertain and stochastic feature and it also can be utilized to solve the sophisticated optimization problem [17], [18], [19]. State-of-the-art offers a good volume of recent works that have utilized DRL for optimizing computation offloading in wireless networks. For instance, the authors in [20] proposed a smart DRL-based resource allocation scheme in wireless networks, to allocate the computing and network resources for reducing service time and balancing resources. The authors in [21] formulated the joint task offloading decision and bandwidth allocation optimization problem and proposed a Deep-Q Network (DQN) based algorithm to solve the optimization problem. The authors in [22] proposed an edge learning-based offloading algorithm where deep learning tasks are defined as computation tasks and they can be offloaded to edge servers to

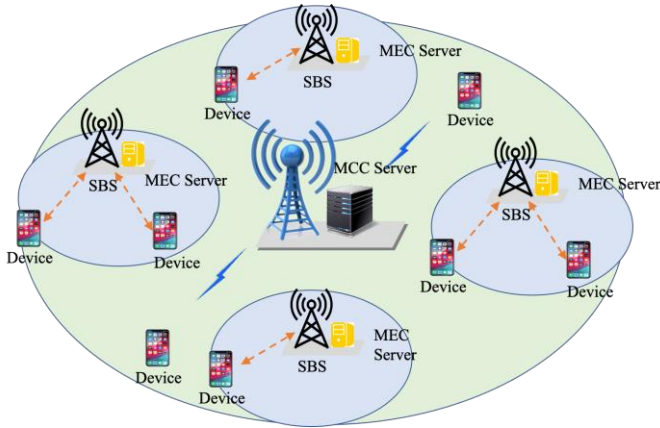


Fig. 1: The scenario of end-edge-cloud heterogeneous networks with multiple users .

improve inference accuracy. The authors in [23] proposed a deep Q-learning based task offloading scheme to select the optimal edge server and the optimal transmission mode to maximize task offloading utility. The authors in [24] proposed a deep Q-network based computation offloading scheme to minimize the long-term cost. The authors in [25] proposed the Double Deep Q-Network (DDQN) based backscatter-aided hybrid data offloading scheme to reduce power consumption in data transmission. The authors in [26] proposed to utilize federated learning to design a distributed data offloading and sharing scheme for a secure network. The above works only decide whether to execute tasks locally or offload to edge servers. However, for an end-edge-cloud network,

computation offloading is a three-tier offloading decision-making problem, and the heterogeneous computation capabilities of devices, edge servers, and the cloud add the complexity of the resource allocation. Thus, it is quite difficult to find the optimal solution of the joint three-tier computation offloading and resource allocation problem in end-edge-cloud networks. On the other hand, the above works are mainly based on discrete action space, and thus are of limited value for the problem with continuous action spaces.

In this paper, we consider an end-edge-cloud orchestrated network consisting of mobile devices, edge computing servers and a cloud server, to jointly optimize computation offloading and scheduling heterogeneous computation resources. Since the action of three-tier offloading decision-making is discrete and the action of heterogeneous computation resource allocation is continuous, we incorporate action refinement in DRL and design a new DRL algorithm to concurrently optimize computation offloading and resources allocation.

III. SYSTEM MODEL

We consider a multi-user end-edge-cloud orchestrated 5G heterogeneous network, which consists of one MBS and M SBSs, as shown in Fig. 1. Let $B = \{b_0, b_1, \dots, b_M\}$ denote the set of all base stations, where b_0 is the MBS. There are N devices randomly distributed under the coverage of the MBS. The set of devices is denoted as $U = \{u_1, \dots, u_N\}$. Each SBS is equipped with an edge server to provide computation TABLE I: Summary of Key Notations

Notation	Definition
B	The set of base stations
U	The set of devices
$D_i = (d_i, c_i, \tau_i)$	Computation task of device i where d_i , c_i and τ_i are input data size, required computation resource, and the upper limit on latency for completing the task, respectively
x_i, y_{ij}, z_{im}	Offloading variables where x_i means local computing, y_{ij} means offloading to SBS, z_{im} means offloading to MBS
R_{ij}^s, R_{i0}^m	Wireless communication data rate between devices and base stations
p_i	Transmission power of device i
h_{ij}^s, h_{i0}^m	Wireless channel gain between devices and base stations
σ^2	Noise power
b_s, b_m	Bandwidth of SBS and MBS, respectively
r_{ij}^s, r_{i0}^m	Distance between devices and base stations
$Til(Eil)$	Task computing time (Energy consumption) of local computing

$T_{ijs}(E_{ijs})$	Task completion time (Energy consumption) for offloading task to SBS
$T_{i0}^m(E_{i0}^s)$	Task completion time (Energy consumption) for offloading task to MBS
f_i^l, F_j^s, F_0^m	The total computation resource of device i , SBS j , and the MBS, respectively.
f_{ij}^s, f_{i0}^m	Variables of computation resource that f_{ij}^s is the computation resource of SBS j assigned to device i and f_{i0}^m is the computation resource of MBS assigned to device i
e_{ij}, e_0	The energy consumption per CPU cycle of SBS j and the MBS, respectively.

resource close to the end devices. The MBS is equipped with a cloud server which has relatively more computation resource. Each device has a computation-intensive and delaysensitive task to be executed within a certain execution deadline, for applications such as augmented/virtual reality, online interactive game, and navigation. Let us denote the task as $D_i = (d_i, c_i, \tau_i)$, where d_i and c_i denote the input data size and the required computation resource, respectively, and τ_i indicates the upper limit on latency for completing the task.

Devices may compute their tasks locally if the task completion delay will be within the limit. Otherwise, devices will offload their tasks to nearby edge servers for edge computing. We consider devices are able to offload their tasks to a nearby

SBS or to the MBS. Let $x_i^l, y_{ij}^s, z_i^m \in \{0,1\}$ denote: task D_i is computed locally, task D_i is offloaded to SBS j , and task D_i is executed at the MBS, respectively. Each device only can choose one way to execute its task, thus offloading variables should satisfy the constraint $x_i^l + \sum_{j=1}^M y_{ij}^s + z_i^m = 1$.

For convenience, the key notations of our system model are summarized in Table I.

A. Communication Model

Devices are associated with the MBS or the SBSs. The wireless communication data rates between devices and base stations can vary depending on transmission power, interference, bandwidth. As shown in Fig. 1, SBSs are deployed at hot spots and they do not have significant coverage overlap. To utilize spectrum efficiently, all SBSs reuse the MBS's frequency resource. Orthogonal Frequency Division Multiple Access (OFDMA) is adopted in wireless networks such that the interference between the MBS and SBSs can be well suppressed [27].

In this work, we consider devices first communicate with the nearest SBS to perform computation offloading. We define γ_j^s as the coverage radius of SBS j . If the distance between device i and SBS j is less than γ_j^s (i.e., $r_{ij}^s < \gamma_j^s$), device i can communicate with SBS j . The wireless communication data rate between device i and SBS j can be expressed as

$$R_{ij}^s = b^s \log_2 \left(1 + \frac{p_i h_{ij}^s (r_{ij}^s)^{-\alpha}}{\sigma^2 + \sum_{i' \in \mathcal{U} \setminus \{i\}, j' \in \mathcal{B} \setminus \{j\}} p_{i'} h_{i'j'}^s (r_{i'j'}^s)^{-\alpha}} \right) \quad (1)$$

where b^s denotes channel bandwidth between device i and SBS j , p_i is the transmission power of device i , h_{ij}^s is the channel gain between device i and SBS j , α is the path loss exponent, and σ^2 is the noise power.

$\sum_{i_0 \in \mathcal{U} \setminus \{i\}, j_0 \in \mathcal{B} \setminus \{j\}} p_{i_0} h_{i_0 j_0}^s (r_{i_0 j_0}^s)^{-\alpha}$ is the interference from other SBSs.

If device i does not lie within the coverage of any SBS, it will communicate with the MBS. The wireless communication data rate between device i and the MBS can be defined as

$$R_{i0}^m = b^m \log_2 \left(1 + \frac{p_i h_{i0}^m (r_{i0}^m)^{-\alpha}}{\sigma^2} \right), \quad (2)$$

where b^m represents the channel bandwidth between device i and the MBS, h_{i0}^m is the channel gain between device i and the MBS, r_{i0}^m is the distance between device i and the MBS.

B. Computation Model

For device i , task D_i can be executed in three ways, i.e., executed locally on the device, offloaded to an SBS, or offloaded to the MBS.

1) *Local Computing*: If device i chooses to compute task D_i locally, the whole computation resource of this device will be utilized to compute the task. We denote f_i^l as the computation resource (i.e., CPU cycles per second) of device i . The local task completion delay only includes the task computing delay, which can be expressed as

$$T_{il} = c_i / f_i^l, \quad (3)$$

The energy consumption of unit computation resource is $\varsigma(f_i^l)^2$, where ς is the effective switched capacitance depending on the chip architecture [7]. We denote the local energy consumption for computing task D_i as E_i^l , which can be defined as

$$E_i^l = \varsigma c_i (f_i^l)^2. \quad (4)$$

2) *Offloading to SBS*: Several devices may be associated with the same SBS and offload their tasks to the SBS simultaneously such that each one of them is only assigned to a part of the computation resource of the SBS. We denote by F_j^s the entire computation resource of SBS j and by f_{ij}^s the computation resource of SBS j allocated to device i for processing task D_i . The total amount of assigned computation resource cannot exceed the entire computation resource of SBS j , i.e., $\sum_{i \in \mathcal{U}} y_{ij}^s f_{ij}^s \leq F_j^s$.

The task completion delay in this case consists of three parts. The first one is the uplink wireless transmission delay for offloading task D_i from device i to SBS j . The second one is the task computing delay which is related to the allocated computation resource f_{ij}^s . The third part is the downlink wireless transmission delay of the computation result from SBS j to device i . Since the size of the result is often much smaller than the input data size, we ignore the duration of the downlink transmission. The task completion delay for offloading task D_i from device i to SBS j can be written

as

$$T_{ij}^s = d_i/R_{ij}^s + c_i/f_{ij}^s, \quad (5)$$

The total energy consumption for completing task D_i is given by

$$E_{ij}^s = p_i d_i/R_{ij}^s + c_i * e_j, \quad (6)$$

where e_j is the energy consumption per computation resource of SBS j .

3) *Offloading to the MBS*: If device i chooses to offload its task to the MBS, the task execution process is similar to the case of offloading to an SBS. Thus, the task completion delay for offloading task D_i from device i to the MBS can be expressed as

$$T_{i0}^m = d_i/R_{i0}^m + c_i/f_{i0}^m, \quad (7)$$

where f_{i0}^m is the computation resource of the MBS assigned to device i . The total amount of assigned computation resource cannot exceed the entire computation resource of the MBS, i.e., $\sum_{i \in \mathcal{U}} z_i^m f_{i0}^m \leq F_0^m$, where F_0^m denotes the entire computation resource of the MBS. The corresponding energy consumption for completing task D_i is $E_{i0}^m = p_i d_i/R_{i0}^m + c_i * e_0$, where e_0 is the energy consumption per computation resource of the MBS.

IV. PROBLEM FORMULATION

The objective of the joint computation offloading and resource allocation problem is to minimize system energy consumption. Taking the constraints of computation resources into account, we formulate the optimization problem as follows:

$$\min \sum_{i \in \mathcal{U}} x_i T_{i0}^m + \sum_{j \in \mathcal{B}} y_{ij} E_{ij}^s + \sum_{i \in \mathcal{U}} z_i E_{i0}^m \quad (8)$$

$$s.t. \sum_{j \in \mathcal{B}} y_{ij} T_{ij}^s + x_i T_{i0}^m + \sum_{i \in \mathcal{U}} z_i T_{i0}^m \leq \tau_i, j \in \mathcal{B} \quad (8a)$$

$$x_i \in \{0, 1\}, i \in \mathcal{U} \quad (8b)$$

$$y_{ij} + x_i + z_i = 1, \quad j \in \mathcal{B} \quad (8c)$$

$$\sum_{i \in \mathcal{U}} y_{ij} f_{ij}^s \leq F_j^s, \sum_{i \in \mathcal{U}} z_i f_{i0}^m \leq F_0^m, \quad (8d)$$

$$x_i^l, y_{ij}^s, z_i^m \in \{0, 1\} \quad (8e)$$

Constraint (8a) guarantees that the task processing delay cannot exceed the upper limit τ_i . Constraints (8b) and (8e) together ensure that each device only chooses one way to compute its task, i.e., executing locally or offloading to an SBS or the MBS. Constraint (8c) ensures that the sum of the computation resources of each base station allocated to all tasks does not exceed the total amount of computation resources the base station has. Because of the integer constraint (8e), problem (8) is NP-hard [28].

We attempt to exploit DRL to design a strategy for minimizing the system energy consumption. Since DRL is based on Markov Decision Process (MDP), we first transform

problem (8) as MDP form and then propose a DRL-based algorithm to solve it.

A typical MDP is defined by a 4-tuple $M = (S, A, P, R)$, where S is a finite set of states, A is a finite set of actions, P represents transition probability from state s to state s^0 after executing action a , and R is an immediate reward function for taking action a .

1) *System State*: At the beginning of each time slot, the MBS observes system states of heterogeneous wireless networks which includes all task offloading requests of devices, the available computation resource of the MBS and SBSs, locations of SBSs and devices, and wireless data rates between devices and base stations. State $s_t \in S$ at time slot t can be defined as $s_t = \{\mathbf{d}(t), \mathbf{c}(t), \tau(t), \mathbf{R}^s(t), \mathbf{R}^m(t), \mathbf{F}(t)\}$, (9) where

- $\mathbf{d}(t) = [d_1(t), \dots, d_N(t)]$: represents input data sizes of computation-intensive tasks at time slot t ;
- $\mathbf{c}(t) = [c_1(t), \dots, c_N(t)]$: represents the remaining required resources for completing the computation-intensive tasks at time slot t ;
- $\tau(t) = [\tau_1(t), \dots, \tau_N(t)]$: represents the maximal delay of each computation-intensive tasks at time slot t ;
- $\mathbf{R}^s(t) = \{[R_{11}^s(t), \dots, R_{N1}^s(t)], \dots, [R_{NM}^s(t), \dots, R_{NM}^s(t)]\}$, represents wireless data rates between devices and SBSs at time slot t ;
- $\mathbf{R}^m(t) = \{[R_{10}^m(t), \dots, R_{N0}^m(t)]\}$, represents wireless data rates between devices and the MBS at time slot t ;
- $\mathbf{F}(t) = [F_0^m(t), F_1^s(t), \dots, F_M^s(t)]$: represents the available computation resource of the MBS and SBSs at time slot t , respectively;

2) *System Action*: The joint computation offloading and resource allocation problem consists of two main components: *offloading decision-making* and *computation resource allocation*. Action $a_t \in A$ in time slot t is defined as $a_t = \{\mathbf{x}(t), \mathbf{y}(t), \mathbf{z}(t), \mathbf{f}^s(t), \mathbf{f}^m(t)\}$, (10)

where

- $\mathbf{x}(t) = [x_1^l(t), \dots, x_N^l(t)]$: represents offloading decisions whether to compute tasks locally, i.e., if $x_i^l(t) = 1$, task D_i will be executed locally at time slot t ;
- $\mathbf{y}(t) = [y_{11}^s(t), \dots, y_{1M}^s(t), \dots, y_{N1}^s(t), \dots, y_{NM}^s(t)]$: denotes offloading decision about whether or not to offload tasks to SBSs, i.e., if $y_{ij}^s(t) = 1$, task D_i will be offloaded to SBS j ;
- $\mathbf{z}(t) = [z_1^m(t), \dots, z_N^m(t)]$: means offloading decision

about whether or not to offload tasks to the MBS, i.e., if $z_i^m(t) = 1$, task D_i will be offloaded and executed at the MBS at time slot t ,

- $\mathbf{f}^s(t) = [f_{11}^s(t), \dots, f_{1M}^s(t), \dots, f_{N1}^s(t), \dots, f_{NM}^s(t)]$: represents the computation resource that each SBS assigns to each device at time slot t .
- $\mathbf{f}^m(t) = [f_{10}^m(t), \dots, f_{N0}^m(t)]$: denotes the computation resource that the MBS assigns to each device at time slot t .

3) *Reward Function*: The objective of joint computation offloading and resource allocation problem is to minimize system energy consumption. After an action a_t is taken, the system computes immediate reward $R^{imm}(s_t, a_t)$ and then updates system state. Thus, the immediate reward function can be defined as

$$R^{imm}(s_t, a_t) = -E[XX_{Xli}(t)E_{il}(t) + y_{ijs}(t)E_{ijs}(t) + z_i^m(t)E_{i0}^m(t)], \quad (11)$$

where $E_i^l(t)$, $E_{ij}^s(t)$, and $E_{i0}^m(t)$ denote local energy consumption for executing task D_i , energy consumption for processing task D_i when offloaded to SBS j , and energy consumption for computing task D_i when offloaded to the MBS, respectively, at time slot t .

4) *Next State*: After computing immediate reward, the system will update state from s_t to s_{t+1} . As the system performs computation offloading and resource allocation in time slot t , the state that needs to be updated is $\mathbf{d}(t), \mathbf{c}(t), \tau(t), \mathbf{F}(t)$.

If device i chooses to compute its task locally (i.e., $x_i^l = 1$), $c_i(t)$ and $\tau_i(t)$ need to be updated based on Eq. (3), the computed task during time slot t is $f_i^l * \Delta t$, where Δt is the length of each time slot. $c_i(t+1) = c_i(t) - f_i^l * t$ and $\tau_i(t+1) = \tau_i(t) - t$. Since local computing does not involve wireless transmission, there is no time computation and energy computation on wireless channels such that we consider $d_i(t) = 0$ in time slot t .

If device i chooses to offload its task to SBS j (i.e., $y_{ijs} = 1$), $d_i(t)$, $c_i(t)$, and $\tau_i(t)$ are updated based on Eq. (5). Specifically, $d_i(t+1) = d_i(t) - R_{ijs} * t$, $c_i(t+1) = c_i(t) - f_{ij}^s * t$, and $\tau_i(t+1) = \tau_i(t) - t$. The available computation resource of each SBS is also updated. Specifically, $F_{js}^s(t+1) =$

$F_{js}^s(t) - P_{i \in U, y_{ijs}^s} * f_{ij}^s$ [29]. If device i chooses to offload its task to the MBS, the updated process is similar to the case of offloading to an SBS.

The objective of the joint computation offloading and resource allocation is to maximize the long-term revenue of the wireless networks, i.e., to maximize the cumulative total reward, given by

$$\mathcal{R} = \max \mathbb{E} \left[\sum_{t=0}^{T-1} \varepsilon^t \mathcal{R}(s_t, a_t) \right], \quad (12)$$

where $\varepsilon \in [0, 1]$ is the discount factor. If all tasks can be completed within their stringent delay constraints, the system agent gets a total reward R . Otherwise, the agent receives a penalty, which is a negative constant.

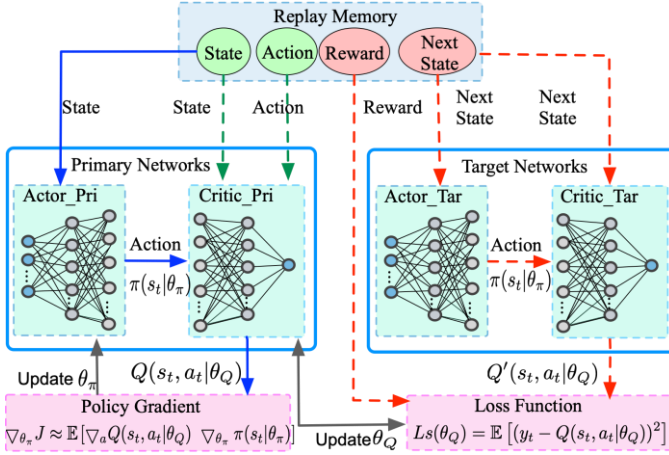
Fig. 2: The structure of DDPG

V. DRL-BASED COMPUTATION OFFLOADING AND RESOURCE ALLOCATION SCHEME

Since the action space of the proposed problem involves continuous variable, we utilize actor-critic based DRL algorithm, Deep Deterministic Policy Gradient (DDPG) to find the solution of MDP and then we incorporate action refinement in DRL to make jointly offloading and resource allocation. In this section, we first introduce the DDPG-based computation offloading and resource allocation algorithm. Then, we describe the way of action refinement.

A. DDPG-based Computation Offloading and Resource Allocation Algorithm

AI is a promising approach to facilitate in-depth feature discovery such that the dynamic input fields for edge computing and caching problems can be obtained in advance. In addition, AI is also a promising tool to tackle complex optimization problems, such as resource allocation. DRL is a branch of AI research where an agent learns by interacting with the environment. Here, we exploit the advanced DRL algorithm-DDPG to solve (12). DDPG is an *Actor-Critic* framework based algorithm, where *Actor* is used to generate actions and *Critic* is used to guide the actor to produce better actions. The DDPG consists of three modules: primary network, target network, and replay memory, as shown in Fig. 2. Primary network generates the detailed computation offloading and resource allocation policy by mapping current state s_t to an action a_t . Primary network consists of two Deep Neural Networks (DNN), namely primary actor DNN $\pi(s_t|\theta_\pi)$ and primary critic DNN $Q(s_t, a_t|\theta_Q)$. Target network is used to generate target values for training primary critic DNN. The structure of target network is the same as the structure of primary network but with different parameters, the target network is denoted by $\pi'(s_t|\theta_\pi')$ and $Q^0(s_t, a_t|\theta_Q')$. Replay memory is used to store experience tuples. Experience tuples include current state s_t , the selected action a_t , reward $R^{imm}(s_t, a_t)$, and the next state s_{t+1} . The random experience fetched from the replay memory breaks up the correlation among the experiences in a mini-batch.



- 8: $\nabla_{\theta_{\pi}} J \approx \mathbb{E} [\nabla_a Q(s_t, a_t | \theta_Q) \nabla_{\theta_{\pi}} \pi(s_t | \theta_{\pi})]$ update θ_{π} by minimizing loss function (17);
- 9: Sample a mini-batch of V tuples from replay memory;
- 10: Compute target value y_t based on (18);
- 11: Update the primary critic DNN network parameter θ_Q by minimizing loss function (17);
- 12: Update the primary actor DNN network parameter θ_{π} using the sampled policy gradient (14);
- 13: Update the target network parameters:

$$\theta_{\pi}^T \leftarrow \omega \theta_{\pi} + (1 - \omega) \theta_{\pi}^T$$

$$\theta_{Q^T} \leftarrow \omega \theta_Q + (1 - \omega) \theta_{Q^T}$$
- 14: end for
- 15: end for

1) Primary Actor DNN Training:

The explored policy can be defined as a function parametrized by θ_{π} , mapping current state to an action $a^* = \pi(s_t | \theta_{\pi})$ where a^* is a proto-actor action generated by the mapping and $\pi(s_t | \theta_{\pi})$ is the explored edge caching and content delivery policy produced by DNN. By adding an Ornstein-Uhlenbeck noise N_t , the constructed action can be described as [30]

$$a_t = \pi(s_t | \theta_{\pi}) + N_t. \quad (13)$$

where Ornstein-Uhlenbeck noise N_t means the random exploration of DDPG.

The primary actor DNN updates network parameter θ_{π} using the sampled policy gradient as,

$$\nabla_{\theta_{\pi}} J \approx \mathbb{E} [\nabla_a Q(s, a | \theta_Q) |_{s=s_t, a=\pi(s_t)} \nabla_{\theta_{\pi}} \pi(s | \theta_{\pi}) |_{s=s_t}], \quad (14)$$

where $Q(s, a | \theta_Q)$ is an action-value function and will be introduced in the following. Specifically, at each training step, θ_{π} is updated by a mini-batch experience $\langle s_t, a_t, R^{imm}_{s_t+1} \rangle$, $t \in \{1, \dots, V\}$, randomly sampled from the replay memory,

$$\theta_{\pi} = \theta_{\pi} - \frac{\alpha_{\pi}}{V} \sum_{t=1}^V [\nabla_a Q(s, a | \theta_Q) |_{s=s_t, a=\pi(s_t)} \nabla_{\theta_{\pi}} \pi(s | \theta_{\pi}) |_{s=s_t}], \quad (15)$$

Algorithm 1 DDPG-based Content Sharing algorithm

- 1: Initialize the primary actor DNN $\pi(s | \theta_{\pi})$ and the primary critic DNN network $Q(s, a | \theta_Q)$ with parameters θ_{π} and θ_Q ;
- 2: Initialize the target actor DNN $\pi^0(s | \theta_{\pi}^T)$ and the target critic DNN network $Q^0(s, a | \theta_Q^T)$ with parameters $\theta_{\pi}^T \leftarrow \theta_{\pi}$ and $\theta_Q^T \leftarrow \theta_Q$; Initialize replay memory;
- 3: for each episode do
- 4: Initialize environment setup;
- 5: for each time slot t do
- 6: Execute action a_t based on (13).
- 7: Observe reward $R^{imm}(s_t, a_t)$ based on (11) and update state s_{t+1} ;
- 8: Store tuple $\langle s_t, a_t, R^{imm}(s_t, a_t), s_{t+1} \rangle$ into where α_{π} is the learning rate of the primary actor DNN.

2) Primary Critic DNN Training:

The primary critic DNN evaluates the performance of the selected action based on the action-value function. The action-value function is calculated by the Bellman optimality equation and can be expressed as

$$Q(s_t, a_t | \theta_Q) = \mathbb{E} [R^{imm}(s_t, a_t) + \varepsilon Q(s_{t+1}, \pi(s_{t+1}) | \theta_Q)], \quad (16)$$

Here, the primary critic DNN takes both current state s_t and next state s_{t+1} as input to calculate $Q(s_t, a_t | \theta_Q)$ for each action.

The primary critic DNN updates the network parameter θ_Q by minimizing the loss function $Ls(\theta_Q)$. The loss function is defined as

$$Ls(\theta_Q) = \mathbb{E} [(y_t - Q(s_t, a_t | \theta_Q))^2], \quad (17)$$

where y_t is the target value and can be obtained by

$$y_t = R^{imm}(s_t, a_t) + \varepsilon Q'(s_{t+1}, \pi'(s_{t+1} | \theta_{\pi}^T) | \theta_Q^T). \quad (18)$$

where $Q'(s_{t+1}, \pi'(s_{t+1} | \theta_{\pi}^T) | \theta_Q^T)$ is obtained through the target network, i.e., the network with parameters θ_{π}^T and θ_Q^T . The gradient of $Ls(\theta_Q)$ is calculated as

$$\nabla_{\theta_Q} Ls = \mathbb{E} [2(y_t - Q(s_t, a_t | \theta_Q)) \nabla_{\theta_Q} Q(s_t, a_t)] \quad (19)$$

At each training step, θ_Q is updated with a mini-batch experiences $\langle s_t, a_t, R^{imm}_{s_t+1} \rangle$, $t \in \{1, \dots, V\}$, that randomly sampled from the replay memory,

$$\theta_Q = \theta_Q - \frac{\alpha_Q}{V} \sum_{t=1}^V [2(y_t - Q(s_t, a_t | \theta_Q)) \nabla_{\theta_Q} Q(s_t, a_t)], \quad (20)$$

where α_Q is the learning rate of the primary critic DNN.

3) Target Network Training:

The target network can be regarded as an old version of the primary network with different parameters θ_{π}^T and θ_Q^T . At each iteration, the parameters θ_{π}^T and θ_Q^T are updated based on the following definition:

$$\theta_{\pi}^T = \omega \theta_{\pi} + (1 - \omega) \theta_{\pi}^T,$$

$$\theta_Q^T = \omega \theta_Q + (1 - \omega) \theta_Q^T, \quad (21)$$

where $\omega \in [0,1]$.

The proposed DDPG-based computation offloading and resource allocation algorithm is summarized as Algorithm 1. The algorithm first initializes the computation offloading and resource allocation policy $\pi(s|\theta_\pi)$ of the primary actor DNN with parameter θ_π , and initializes the action-value function $Q(s_t, a_t|\theta_Q)$ of the primary critic DNN network with parameter θ_Q . Both parameters θ_π^T and θ_Q^T of the target network are also initialized. Then, according to current policy $\pi(s|\theta_\pi)$ and state s_t , the primary actor DNN generates action a_t based on (13). According to the observed reward $R^{imm}(s_t, a_t)$ and next state s_{t+1} , a tuple $\langle s_t, a_t, R^{imm}(s_t, a_t), s_{t+1} \rangle$ is constructed and stored into replay memory. Note that, if replay memory is going to be full, the oldest experience will be deleted to make room for the latest one. Based on minibatch technique, the algorithm updates the primary critic DNN network by minimizing the function $Ls(\theta_Q)$ and updates the primary actor DNN using the sampled policy gradient. After a period of training, the parameters of the target networks are updated based on Eq. (21).

The complexity of Algorithm 1 is mainly determined by four neural networks and one activation layer. Assuming that Actor DNN contains L fully connected layers and Critic DNN contains K fully connected layers. The time complexity can be calculated as [31]

$$\begin{aligned} & 2 \times \sum_{l=0}^L n_{Actor,l} \cdot n_{Actor,l+1} + 2 \times \sum_{k=0}^K n_{Critic,k} \cdot n_{Critic,k+1} \\ & = O\left(\sum_{l=0}^L n_{Actor,l} \cdot n_{Actor,l+1} + \sum_{k=0}^K n_{Critic,k} \cdot n_{Critic,k+1}\right) \end{aligned} \quad (22)$$

where $n_{Actor,l}$ and $n_{Critic,k}$ mean the unit number in the l -th Actor DNN layer and the k -th Critic DNN layer, $n_{Actor,0}$ and $n_{Critic,0}$ equal the input size.

B. Action Refinement

The outputs of DDPG-based algorithm are continuous values. However, the actions about offloading decisions should be integer values (i.e., x_i^l, y_{ij}^s and z_i^m). Therefore, we adopt a rounding technique to refine these actions. The rounding technique consists of the following three steps: 1) obtain the continuous solution from a_t , 2) construct a weighted bipartite graph to establish the relationship between devices and base stations, 3) find an integer matching to obtain the integer solution.

1) *Offloading decisions abstraction*: We define $\mathbf{w}(t)$, $[\mathbf{x}(t), \mathbf{y}(t), \mathbf{z}(t)]$, thus each element $w_{ij}(t)$ in $\mathbf{w}(t)$ is a continuous value.

2) *Bipartite graph construction*: We apply rounding technique [32] to transform continuous value into integers by constructing the weighted bipartite graph $G(U, V, E)$ to

establish the relationship between devices and their offloading strategies. U is one side of bipartite graph which represents devices in the network. $V = \{v_{js} : j = 0, 1, \dots, M, M+1, s = 1, \dots, J_j\}$ is the other side of bipartite graph which is a set of virtual nodes. The index j of v_{js} is related to offloading strategies, where $j = 0$ means local computing, $j = \{1, \dots, M\}$ means offloading to SBS, and $j = M+1$ means offloading to the MBS. The index of s ranges from 1

to J_j , where $J_j = \lceil \sum_{i=1}^N w_{ij}(t) \rceil$ means there are J_j devices will choose the j -th offloading strategy.

The most important procedure for constructing graph G is to set the edges and the edge weight between U and V . The edges in G are constructed using Algorithm 2. Specifically, if $J_j \leq 1$, there is only one node v_{j1} corresponding to base station j . In this case, for each $w_{ij}(t) > 0$, we add an edge between node u_i and v_{j1} , and set the weight of this edge as $w_{ij}(t)$. Otherwise, for each $s \in \{1, 2, 3, \dots, J_j - 1\}$, we need to find the minimum index i_s which satisfies $\sum_{i=1}^{i_s} w_{ij}(t) \geq s$. If $s = 1$, we set $i_{s-1} = 0$. For each $i \in \{i_{s-1} + 1, \dots, i_s - 1\}$ and $w_{ij}(t) > 0$, we add (u_i, v_{js}) into E and set the weight of this edge as $w_{ij}(t)$. If $i = i_s$, we add edge (u_i, v_{js}) into

E and set the weight as $s - \sum_{i=1}^{i_s-1} w_{ij}(t)$. This ensures that the sum of weights for all edges (u_i, v_{js}) is equal to s . If Algorithm 2 Construct the edges of bipartite graph G

```

1: Set  $E \leftarrow \emptyset$ .
2: if  $J_j \leq 1$  then
3:   There is only one node  $v_{j1}$  corresponding to base station  $j$ .
4:   for each  $w_{ij}(t) > 0$  do
5:     Add edge  $(u_i, v_{j1})$  into  $E$  and set  $e_{ij1} = w_{ij}(t)$ .
6:   end for
7: else
8:   for each  $s \in \{1, 2, \dots, J_j - 1\}$  do
9:     Find the minimum index  $i_s$  where  $\sum_{i=1}^{i_s} w_{ij}(t) \geq s$ .
10:    if  $s = 1$  then
11:       $i_{s-1} = 0$ 
12:    end if
13:    for  $i \in \{i_{s-1} + 1, \dots, i_s - 1\}$  and  $w_{ij}(t) > 0$  do
14:      Add edge  $(u_i, v_{js})$  into  $E$  with  $e_{ijs} = w_{ij}(t)$ .
15:    end for
16:    if  $i = i_s$  then
17:      Add edge  $(u_i, v_{js})$  into  $E$  with  $e_{ijs} = s - \sum_{i=1}^{i_s-1} w_{ij}(t)$ .
18:    end if
19:    if  $\sum_{i=1}^{i_s} w_{ij}(t) > s$ . then
20:      Add edge  $(u_i, v_{j(s+1)})$  into  $E$  with weight  $e_{ij(s+1)} = \sum_{i=1}^s w_{ij}(t) - s$ .

```

21: end if
 22: end for
 23: end if

$\sum_{i=1}^{i_s} w_{ij}(t) > s$, we add one more edge $(u_i, v_{j(s+1)})$ and set the weight of this edge as $\sum_{i=1}^{i_s} w_{ij}(t) - s$.

3) *Action refinement*: We utilize the Hungarian algorithm [33] to find a complete max-weighted bipartite matching M_{match} . According to the M_{match} , we obtain the integer offloading decisions. Specifically, if (u_i, v_{js}, e_{ijs}) is in the M_{match} , we set $w_{ij}(t) = 1$; otherwise, $w_{ij}(t) = 0$. Based on the definition $\mathbf{w}(t) = [\mathbf{x}(t), \mathbf{y}(t), \mathbf{z}(t)]$, we can decide the values of $x_i^l(t)$, $y_{ij}^s(t)$. Thus, we get the binary offloading policy. The complexity of action refinement algorithm is polynomial in the number of nodes and edges, that is $O(|V||E|)$.

VI. NUMERICAL RESULTS

In this section, we use Python and TensorFlow to evaluate the performance of our proposed DDPG-inspired computation offloading and resource allocation algorithm based on a realword dataset.

A. Simulation Setup

We consider a network topology with one MBS, $M = 10$ SBSs, and $N = 100$ devices. The maximum transmission power of devices is set to 100 mW. The channel gain models presented in 3GPP standardization are adopted here [34]. The noise power is $\sigma^2 = 10^{-11}$ mW. The bandwidth of the MBS and the SBSs are 10 MHz and 5 MHz, respectively. In addition, $e_j = e_0 = 1$ W/GHz [35]. The CPU computation capabilities of devices, SBS, and MBS are 0.5, 10, and 50

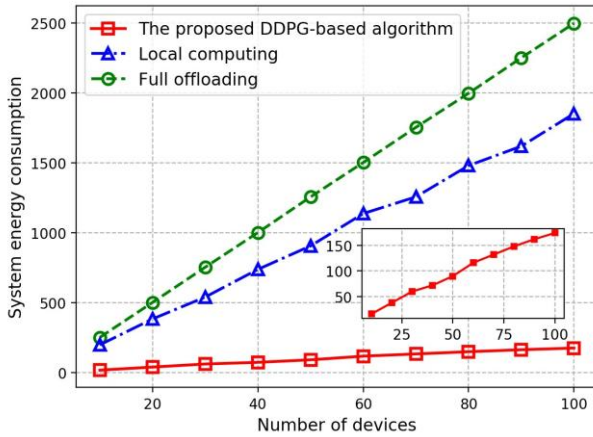


Fig. 3: Comparison of system energy consumption with respect to the number of devices under different schemes.

GHz, respectively. Each device has a computation task. Both the data size of each task and required number of CPU cycles per bit follow the uniform distribution with $U[5,50]$ MB and $U[0.5,5]$ GHz, respectively. The upper limit on latency is set as 1 s.

Our proposed DDPG-inspired algorithm to the above pattern recognition tasks is deployed on a MacBook Pro laptop, powered by two Intel Core i5 processor (clocked at 2.6GHz). The activation function of the DDPG is $\frac{\tanh(x)+1}{2}$. The size of mini-batch is set as 32. The maximum number of episodes is 6000 and the maximum number of steps in each episode is set to 20. The penalty is $100 * N$, where N is the number of devices.

To verify the performance of our proposed algorithm, we introduce the following two benchmark policies:

- *Local computing*: Each mobile user executes its computation tasks locally. The feasible computation resource of each task can be obtained as $f_i^k = \frac{F_i d_i^k c_i^k}{\sum_{k=1}^K d_i^k c_i^k}$. If $\sum_{k=1}^K \frac{d_i^k c_i^k}{f_i^k} \leq T_i^{max}$, all tasks are always locally executed.
- *Full offloading*: All computation tasks are offloaded to the MBS for remote computing. In this policy, all mobile users are associated with the MBS.

TABLE II: Different Types of Tasks

	Data size	Required CPU cycles	Types
Type 1	50 MB	5 GHz	large data size computation-intensive
Type 2	50 MB	0.5 GHz	large data size non-computation-intensive
Type 3	5 MB	5 GHz	small data size non-computation-intensive

B. Performance Analysis

In this subsection, we compare the proposed DDPG-based algorithm with other two benchmarks. In Fig. 3, we plot the energy consumption with respect to the number of devices under different schemes. To evaluate the performance of different algorithms, we consider three types of tasks, as shown in Table II. Fig. 4 shows the energy consumption of each device with respect to different task types under different algorithms.

From Fig. 3, we can draw several observations. First, the proposed DDPG-based algorithm significantly outperforms the two benchmark policies by jointly optimizing offloading decisions and computation resources. Specifically, compared to the local computing policy, the proposed algorithm can offload tasks to edge servers with a relatively low computation

energy consumption. Compared to the full offloading policy, the proposed algorithm ensures offloading each task to the nearby edge node which results in lower energy consumption for wireless transmission. Second, the system energy consumption of local computing increases rapidly as the number of devices becomes large. Third, full offloading policy performs the worst. Since all devices offload their tasks to the MBS. Full offloading policy results in a congested wireless uplink and a higher wireless transmission energy consumption.

From Fig. 4, we can see that when tasks are computationintensive (i.e., type 1 and type 3), the energy consumption of the proposed algorithm is the lowest. The reason is that utilizing the proposed algorithm, computation-intensive tasks can intelligently be offloaded to nearby edge servers. Compared to the scheme of local computing, the energy consumption on each device is dramatic reduced. Compared to the scheme of full offloading, the proposed algorithm can support more offloading action such that the energy consumption on each device can be decreased. When tasks are type 2 (i.e., large data size but less computation resources required), local computing policy results in the lowest energy consumption. This is because transmitting large data incurs higher energy consumption. However, for local computing, the only energy consumption is due to computation. In this case, our proposed algorithm leads to a slightly higher energy consumption than local computing, but it performs much better than full offloading. Overall, our proposed algorithm keeps a relatively low energy consumption for all types of tasks. Thus, we can conclude that the proposed algorithm is the most robust one, considering that a real network will have mixed traffic.

C. Impact of Factors on Performance

In this subsection, we evaluate the impact of different factors on the performance of the proposed DDPG-based algorithm. The factors are: 1) learning rates of actor DNN and critic DNN, 2) discount factor ε , and 3) number of devices. For ease of description, we adopt normalized cumulative reward in the y-axis. Here, a large normalized cumulative reward means a better performance.

To evaluate the impact of the learning rates, we set both α_π and α_Q as $\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$, respectively. Fig. 5 depicts the normalized cumulative reward of the proposed DDPG-based algorithm under learning rates. Since only when α_π and α_Q are equal to 10^{-3} or 10^{-4} the proposed DDPGbased algorithm achieves convergence, we only show the results of these cases in Fig. 5. We can draw several observations

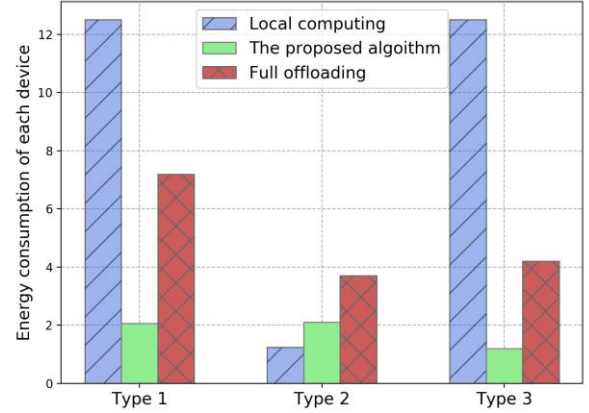


Fig. 4: Comparison of energy consumption on each device with respect to different task types.

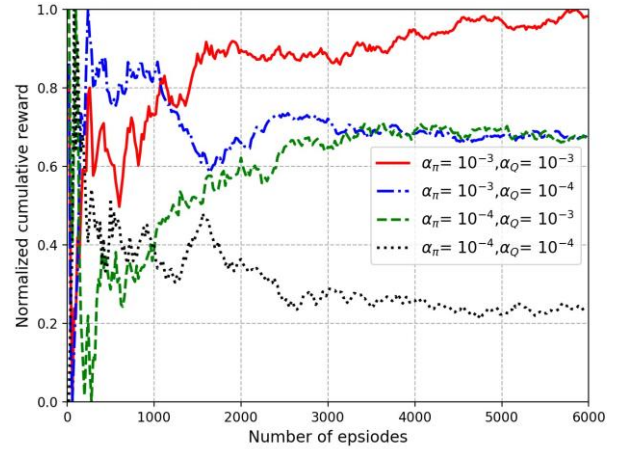


Fig. 5: Impact of learning rate

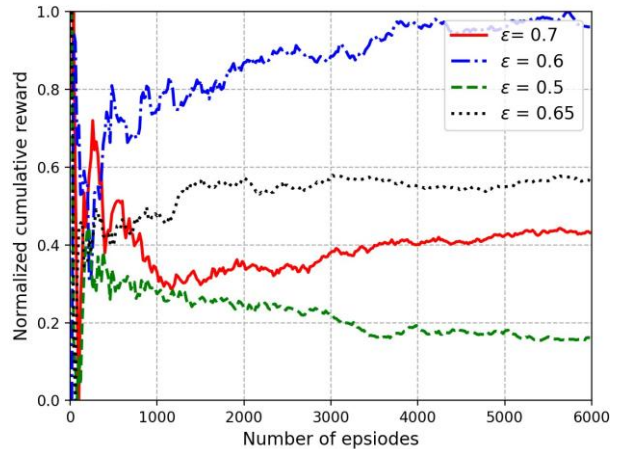


Fig. 6: Impact of discount factor

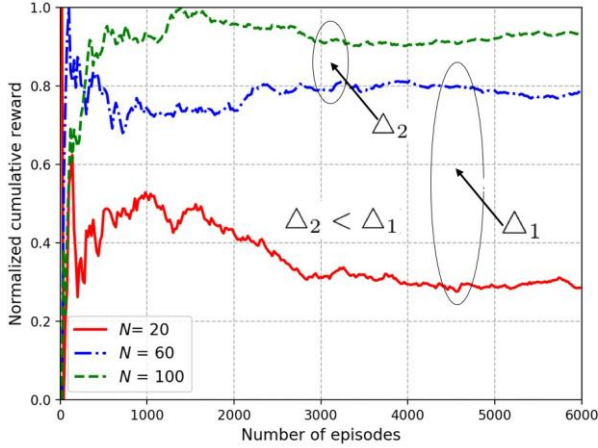


Fig. 7: Impact of number of devices

from Fig. 5. First, when $\alpha_\pi = 10^{-3}$, the reward of red line (i.e., $\alpha_Q = 10^{-3}$) is greater than the reward of blue line (i.e., $\alpha_Q = 10^{-4}$). When $\alpha_\pi = 10^{-4}$, the reward of green line (i.e., $\alpha_Q = 10^{-3}$) is greater than the reward of black line (i.e., $\alpha_Q = 10^{-4}$). Similar, when $\alpha_Q = 10^{-3}$, the reward of red line (i.e., $\alpha_\pi = 10^{-3}$) is greater than the reward of green line (i.e., $\alpha_\pi = 10^{-4}$). When $\alpha_Q = 10^{-4}$, the reward of blue line (i.e., $\alpha_\pi = 10^{-3}$) is greater than the reward of black line (i.e., $\alpha_\pi = 10^{-4}$). Thus, when $\alpha_\pi = 10^{-3}$ and $\alpha_Q = 10^{-3}$, the performance of the proposed algorithm is the best. Second, when $\alpha_\pi = 10^{-3}$ and $\alpha_Q = 10^{-4}$, and $\alpha_\pi = 10^{-4}$ and $\alpha_Q = 10^{-3}$, the proposed algorithm eventually converges to the same value. That means α_π and α_Q have the same influence on the convergence. Third, when $\alpha_Q = 10^{-3}$, the convergence trend of the red solid line is similar to that of the green dash line. When $\alpha_Q = 10^{-4}$ the convergence trend of the blue dash-dot line and the black dot line is similar. Thus, the critic learning rate α_Q considerably influences the convergence trend.

To evaluate the impact of discount factors, we set ε from 0.5 to 0.7. Fig. 6 plots the normalized cumulative reward of the proposed algorithm under different discount factors. First, the proposed algorithm converges irrespective of the values of discount factor chosen. Second, when $\varepsilon = 0.6$, the normalized cumulative reward is clearly higher than the cases when $\varepsilon = 0.65$ and $\varepsilon = 0.7$, which implies that a small discount factor results in a better performance. However, the performance when $\varepsilon = 0.65$ is also better than that when $\varepsilon = 0.5$. Thus, we can conclude that $\varepsilon = 0.6$ is the best discount factor for the proposed algorithm. In fact, since $\mathcal{R} = \sum_{t=0}^{T-1} \varepsilon^t \mathcal{R}(s_t, a_t)$, the discount factor determines the relative ratio of future immediate reward versus current immediate reward. Specifically, rewards received at the t th time slot in the future are discounted exponentially by a factor of ε^t . Note that if we set $\varepsilon = 0$, only current immediate reward is considered. As we

set ε closer to 1, future immediate rewards are given greater weight relative to the current immediate reward.

Fig. 7 shows the impact of the number of devices on the normalized cumulative reward. Here, we set N as $\{20, 60, 100\}$, respectively. We can see that the proposed algorithm converges in all cases and the normalized cumulative reward increases with the increase in N . The growth in the number of devices leads to more offloading requests, which results in the need and consumption of more communication resource and computation resource. Moreover, the gap between the normalized cumulative reward when $N = 100$ and the normalized cumulative reward when $N = 60$ is greater than the gap between the normalized cumulative reward when $N = 60$ and the normalized cumulative reward when $N = 20$ (i.e., $\Delta_2 < \Delta_1$), which means the increment of the normalized cumulative rewards becomes smaller. This observation implies that the gain offered by the proposed algorithm in terms of reducing system energy consumption, is more pronounced when the number of devices is high.

TABLE III: Training Delay of the Proposed Algorithm

Delay(s) \ N	20	40	60
Episodes			
3000	35.17	38.31	51.40
6000	71.35	77.87	100.18

Table III shows the training delay of the proposed algorithm, where N is the number of devices. The number of devices determines the scale of state space and action space. Thus, as the number of devices increases, the training delay increases dramatically. Further, with the increase in the number of episodes, the training delay also increases.

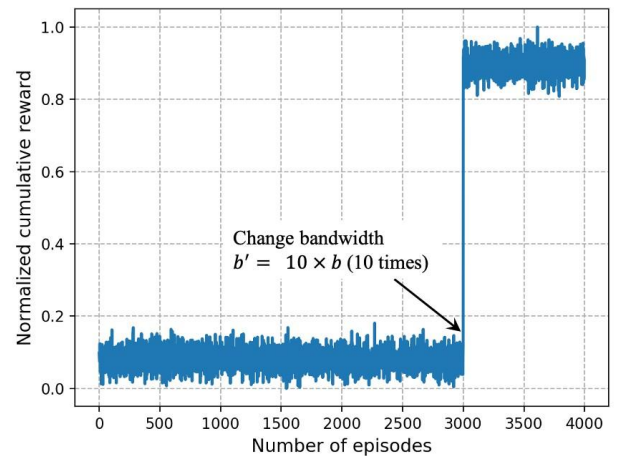


Fig. 8: Convergence under time-varying bandwidth

In Fig. 8, we study the effect of time-varying parameter on the convergence of the proposed algorithm. To quickly verify the effect of time-varying parameters on the proposed algorithm, we consider a small-scale network with 10 devices and 5 SBSs. The bandwidth of the MBS and the SBSs are 2 MHz

and 1 MHz, respectively. After running 3000 episodes, we change the bandwidth of base stations. Specifically, we enlarge the original bandwidth ten times. As shown in Fig. 8, the normalized cumulative reward sharply increases after the bandwidth is changed, and stabilizes at a higher value. This means the proposed algorithm automatically updates its offloading policy and converges to a new optimal solution. Thus, the proposed algorithm can deal with time-varying network environment.

VII. CONCLUSIONS

In this paper, we proposed a joint computation offloading and resource allocation scheme for minimizing system energy consumption in 5G heterogeneous networks. We first presented a multi-user end-edge-cloud orchestrated network in which all devices and base stations have computation capabilities. Then, we formulated the joint computation offloading and resource allocation problem as an optimization problem and transformed it into the form of an MDP. We proposed a DDPG-based algorithm to intelligently minimize system energy consumption by interacting with the environment. Numerical results based on a real-world dataset demonstrated that the proposed DDPG-based algorithm significantly outperforms the benchmark policies in terms of system energy consumption. Extensive simulations indicated that the learning rate, discount factor, and number of end user devices have considerable influence on the performance of the proposed DDPG-based algorithm.

REFERENCES

- [1] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 450–465, 2018.
- [2] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017.
- [3] T. G. Rodrigues, K. Suto, H. Nishiyama, N. Kato, and K. Temma, "Cloudlets activation scheme for scalable mobile edge computing with transmission power control and virtual machine migration," *IEEE Transactions on Computers*, vol. 67, no. 9, pp. 1287–1300, 2018.
- [4] J. Zhao, Q. Li, Y. Gong, and K. Zhang, "Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 8, pp. 7944–7956, 2019.
- [5] C. You, K. Huang, H. Chae, and B.-H. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Transactions on Wireless Communication*, vol. 16, no. 3, pp. 1397–1411, 2017.
- [6] T. T. Nguyen, L. Le, and Q. Le-Trung, "Computation offloading in mimo based mobile edge computing systems under perfect and imperfect csi estimation," *IEEE Transactions on Services Computing*, pp. 1–1, 2019.
- [7] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint computation offloading and user association in multi-task mobile edge computing," *IEEE Transactions on Vehicular Technology*, vol. 67, no. 12, pp. 12313–12325, 2018.
- [8] H. Lu, X. He, M. Du, X. Ruan, Y. Sun, and K. Wang, "Edge qoe: Computation offloading with deep reinforcement learning for internet of things," *IEEE Internet of Things Journal*, 2020.
- [9] Y. Dai, D. Xu, S. Maharjan, G. Qiao, and Y. Zhang, "Artificial intelligence empowered edge computing and caching for internet of vehicles," *IEEE Wireless Communications*, vol. 26, no. 3, pp. 12–18, 2019.
- [10] T. K. Rodrigues, K. Suto, H. Nishiyama, J. Liu, and N. Kato, "Machine learning meets computation and communication control in evolving edge and cloud: Challenges and future perspective," *IEEE Communications Surveys & Tutorials*, 2019.
- [11] K. Zhang, Y. Zhu, S. Maharjan, and Y. Zhang, "Edge intelligence and blockchain empowered 5g beyond for the industrial internet of things," *IEEE Network*, vol. 33, no. 5, pp. 12–19, 2019.
- [12] H. Guo, J. Liu, J. Zhang, W. Sun, and N. Kato, "Mobile-edge computation offloading for ultradense iot networks," *IEEE Internet of Things Journal*, vol. 5, no. 6, pp. 4977–4988, 2018.
- [13] T. G. Rodrigues, K. Suto, H. Nishiyama, and N. Kato, "Hybrid method for minimizing service delay in edge cloud computing through vm migration and transmission power control," *IEEE Transactions on Computers*, vol. 66, no. 5, pp. 810–819, 2016.
- [14] Z. Zhao, R. Zhao, J. Xia, X. Lei, D. Li, C. Yuen, and L. Fan, "A novel framework of three-hierarchical offloading optimization for mec in industrial iot networks," *IEEE Transactions on Industrial Informatics*, vol. 16, pp. 5424–5434, 2020.
- [15] F. Wang, J. Xu, and Z. Ding, "Multi-antenna noma for computation offloading in multiuser mobile edge computing systems," *IEEE Transactions on Communications*, vol. 67, no. 3, pp. 2450–2463, 2018.
- [16] A. Samanta and Z. Chang, "Adaptive service offloading for revenue maximization in mobile edge computing with delay-constraint," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 3864–3872, 2019.
- [17] K. Ahmed, H. Tabassum, and E. Hossain, "Deep learning for radio resource allocation in multi-cell networks," *IEEE Network*, 2019.
- [18] Y. Dai, D. Xu, K. Zhang, S. Maharjan, and Y. Zhang, "Deep reinforcement learning and permissioned blockchain for content caching in vehicular edge computing and networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 4312–4324, 2020.
- [19] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 4298–4311, 2020.
- [20] J. Wang, L. Zhao, J. Liu, and N. Kato, "Smart resource allocation for mobile edge computing: A deep reinforcement learning approach," *IEEE Transactions on emerging topics in computing*, 2019.
- [21] L. Huang, X. Feng, C. Zhang, L. Qian, and Y. Wu, "Deep reinforcement learning-based joint task offloading and bandwidth allocation for multiuser mobile edge computing," *Digital Communications and Networks*, vol. 5, no. 1, pp. 10–17, 2019.
- [22] B. Yang, X. Cao, X. Li, C. Yuen, and L. Qian, "Lessons learned from accident of autonomous vehicle testing: An edge learning-aided offloading framework," *IEEE Wireless Communications Letters*, 2020.
- [23] K. Zhang, Y. Zhu, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Deep learning empowered task offloading for mobile edge computing in urban informatics," *IEEE Internet of Things Journal*, 2019.
- [24] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, "Performance optimization in mobile-edge computing via deep reinforcement learning," in *2018 IEEE 88th Veh. Technol. Conf. (VTC-Fall)*, Aug 2018, pp. 1–6.
- [25] Y. Xie, Z. Xu, J. Xu, S. Gong, and Y. Wang, "Backscatter-aided hybrid data offloading for mobile edge computing via deep reinforcement learning," in *International Conference on Machine Learning and Intelligent Communications*. Springer, 2019, pp. 525–537.
- [26] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and federated learning for privacy-preserved data sharing in industrial iot," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, pp. 4177–4186, 2019.
- [27] J. Xiao, C. Yang, A. Anpalagan, Q. Ni, and M. Guizani, "Joint interference management in ultra-dense small-cell networks: A multidomain coordination perspective," *IEEE Transactions on Communication*, vol. 66, no. 11, pp. 5470–5481, Nov 2018.
- [28] M. R. Garey and D. S. Johnson, *Computers and Intractability; A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman & Co., 1990.
- [29] J. Li, H. Gao, T. Lv, and Y. Lu, "Deep reinforcement learning based computation offloading and resource allocation for mec," in *2018 IEEE Wirel. Commun. and Netw. Conf. (WCNC)*, April 2018, pp. 1–6.
- [30]